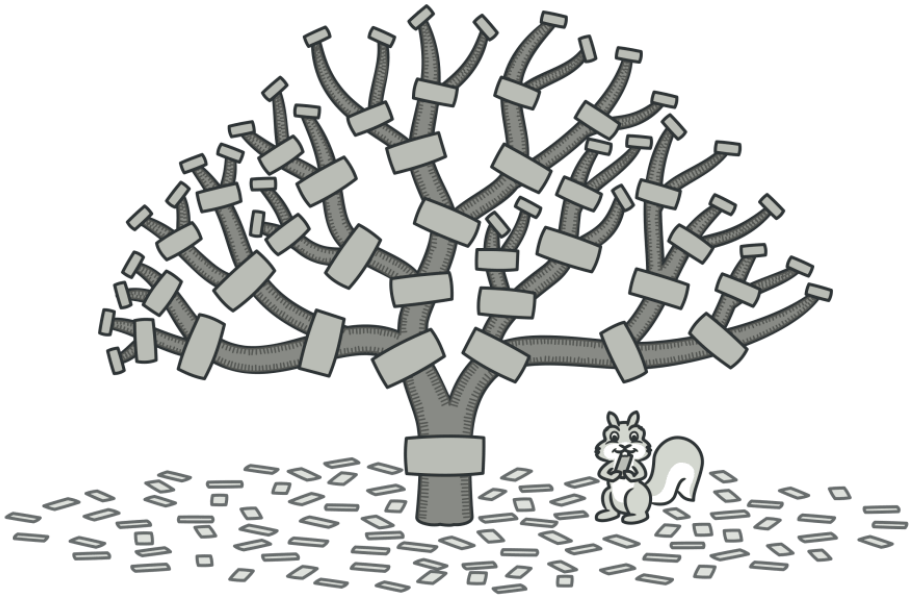
Compositie



Inhoudsopgave

Compositie 1

Vragen 1

Wat is C++ compositie 2

Voor- en nadelen van compositie 3

Voorbeelden 3

Fout 3

Eind vraag – betekenis in de c++ 3

Bronnenlijst 3

Rolverdeling 3

Intro

Intro

Bij het ontwerpen van software in C++ is een van de eerste vragen: hoe zorg ik dat mijn klassen goed met elkaar samenwerken? Een van de belangrijkste manieren om klassen te ontwerpen is compositie. Hierbij bestaat een object uit andere objecten, die als

onderdeel (member) in die klasse zitten. Een auto heeft een motor, een mens heeft een hart en een creature in een spel heeft een positie. De onderdelen horen bij één object en worden door dat object beheerd. Zodra het object verdwijnt, verdwijnen de onderdelen ook.

Het doel is dat de lezer na afloop begrijpt wat compositie is, waarom het in C++ vaak de voorkeur heeft boven andere manieren van klassen koppelen, en wanneer je het wel of niet gebruikt.

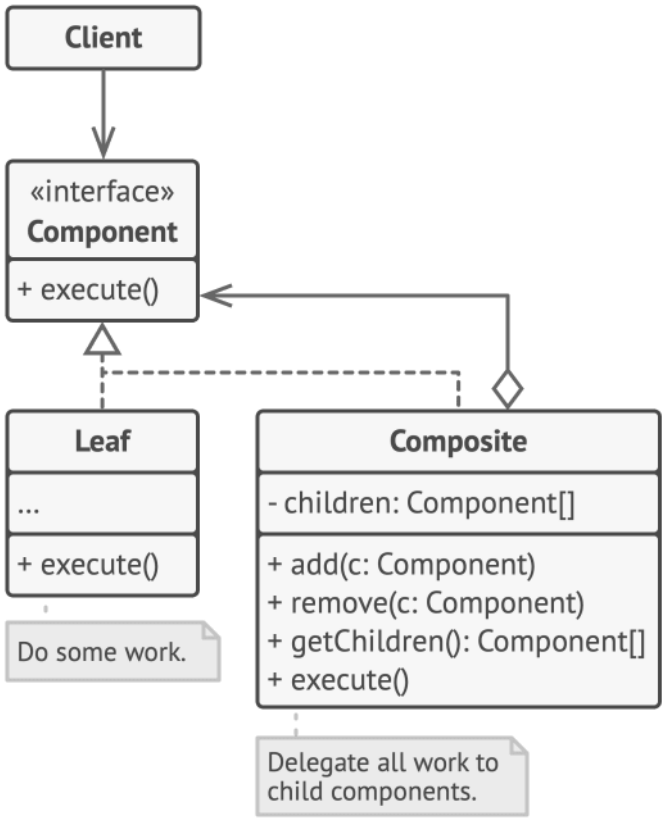
Vragen

Wat is C++ compositie

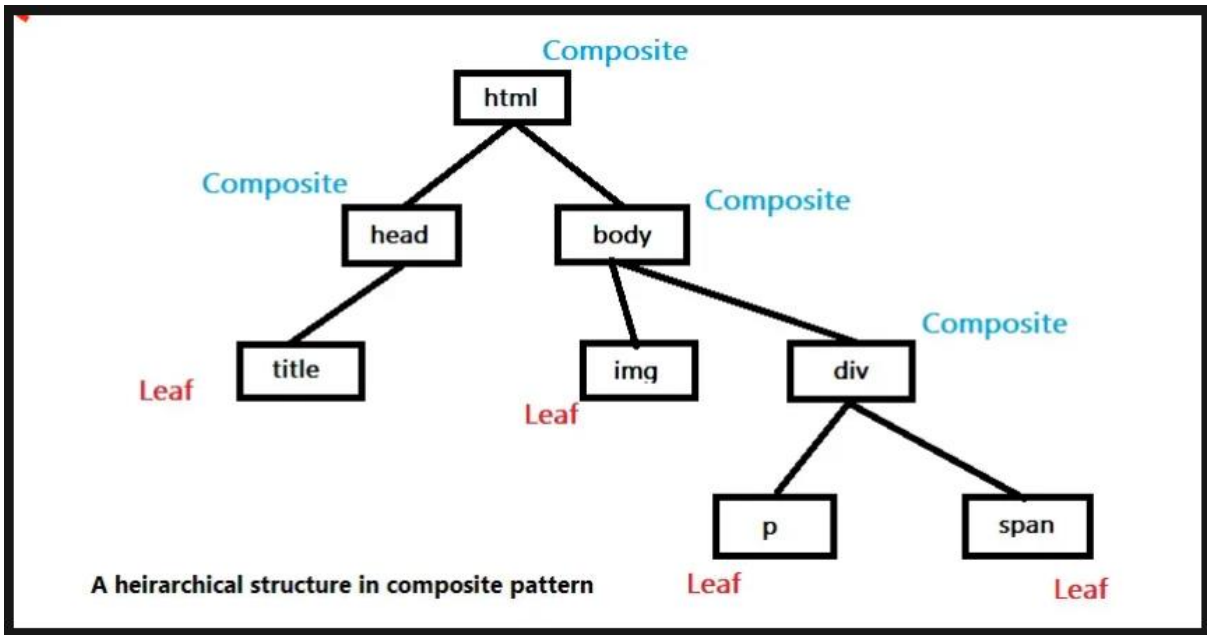
Compositie is een concept in objectgeoriënteerd programmeren waarbij een class een andere class als onderdeel van zichzelf bevat. Het vertegenwoordigt een has-a-relatie tussen objecten. Een auto heeft bijvoorbeeld een motor, of een robot heeft een motor controller.

Bij compositie is het bevattende object de eigenaar van het bevatte object. Dit betekent dat de lifetime van het bevatte object gekoppeld is aan de lifetime van zijn eigenaar. Wanneer de eigenaar wordt aangemaakt, wordt het bevatte object ook aangemaakt. Wanneer de eigenaar wordt vernietigd (destroyed), wordt het bevatte object ook vernietigd.

Compositie wordt gebruikt om complexe systemen op te bouwen uit kleinere, meer gerichte componenten. In plaats van alle functionaliteit in één grote class te plaatsen, kunnen verantwoordelijkheden worden verdeeld over meerdere classes die samenwerken. Dit maakt code gemakkelijker te begrijpen, te onderhouden en te hergebruiken.



(refactoring.guru)



(modern.com)

Voor- en nadelen van compositie

Voordelen:

Hiërarchieondersteuning: Het patroon ondersteunt hiërarchieën van objecten, waardoor we complexe structuren zoals bomen kunnen weergeven.

Uniforme interface: Zowel losse objecten als groepen objecten worden op dezelfde manier behandeld, wat de client-code vereenvoudigt.

Uitbreidbaarheid: Het patroon maakt het eenvoudig om nieuwe typen componenten toe te voegen zonder bestaande code te wijzigen, in lijn met het Open-Closed Principle.

Vereenvoudigde client-code: Clients kunnen eenvoudige en complexe elementen op dezelfde manier behandelen, waardoor er minder complexe if-statements nodig zijn om te controleren of een element een leaf of een composite is.

Nadelen:

Toegenomen complexiteit: Hoewel het Composite-patroon de client-code vereenvoudigt, kan het de totale complexiteit van het systeem vergroten door de extra lagen abstractie.

Overhead: Het beheren van child-componenten kan overhead met zich meebrengen, vooral bij grote structuren, omdat het geheugengebruik kan toenemen. (In deze context gebeurt dit vooral als er te veel tussenstappen zijn om bij de functie te komen die je wilt aanroepen.)

Moeilijk een gemeenschappelijke interface te bieden: Het kan lastig zijn om een gemeenschappelijke interface te maken voor classes waarvan de functionaliteit te sterk verschilt. In sommige situaties zou je de component-interface te veel moeten generaliseren, waardoor deze moeilijker te begrijpen wordt.

Nadelen van het niet toepassen van compositie

Geen child-elementen: De structuur ondersteunt geen nesting, wat betekent dat er geen child-elementen kunnen worden toegevoegd aan een bestaand element zoals een .

Inefficiëntie: Als we willen weten of een element children heeft, of als we child-elementen willen toevoegen, moeten we deze relaties handmatig doorlopen en beheren. Dit kan leiden tot kwadratische tijdscomplexiteit ($O(n^2)$).

Code-duplicatie: Het toevoegen, renderen en beheren van children voor verschillende HTML-elementen zou betekenen dat je code moet dupliceren over meerdere classes.

Gebrek aan hiërarchie: In onze eenvoudige array is er geen hiërarchie. Elk element staat op zichzelf en er is geen manier om verzamelingen van elementen als één object te behandelen.

Moeilijk om relaties te beheren: Als je children wilt toevoegen aan een DOM-element, zou je handmatig moeten bijhouden welk element een parent is en welk element een child is.

Zoektijd: Het zoeken naar child-elementen of het doorlopen van de boomstructuur kan kwadratische tijd kosten, vooral bij diep geneste structuren.

Voorbeelden

Veel games en simulaties hebben creatures of objecten die zich over een bord, kaart of scherm bewegen. Wat al deze creatures/objecten gemeen hebben, is dat ze allemaal een locatie hebben. In dit voorbeeld gaan we een creature-class maken die een point-class gebruikt om de locatie van de creature bij te houden.

Laten we eerst de point-class ontwerpen. Onze creature leeft in een 2D-wereld, dus onze point-class heeft 2 dimensies, X en Y. We gaan ervan uit dat de wereld bestaat uit discrete vakjes, dus deze dimensies zijn altijd integers.

Point2D.h:

```
#ifndef POINT2D_H
#define POINT2D_H

#include <iostream>

class Point2D
{
private:
    int m_x;
    int m_y;

public:
    // A default constructor
    Point2D()
        : m_x{ 0 }, m_y{ 0 }
    {
    }

    // A specific constructor
    Point2D(int x, int y)
        : m_x{ x }, m_y{ y }
    {
    }

    // An overloaded output operator
    friend std::ostream& operator<<(std::ostream& out, const Point2D& point)
    {
        out << '(' << point.m_x << ", " << point.m_y << ')';
        return out;
```

```
    }

    // Access functions
    void setPoint(int x, int y)
    {
        m_x = x;
        m_y = y;
    }

};

#endif
```

Note that because we've implemented all of our functions in the header file (for the sake of keeping the example concise), there is no Point2D.cpp.

Fout

Een van de dingen waar compositie minder geschikt voor is, heeft te maken met het hergebruiken van child-objecten op meerdere plaatsen. Bij compositie hoort een child-object bij precies één parent-object. Als het parent-object wordt verwijderd, worden de child-objecten meestal ook verwijderd. Daardoor kunnen deze objecten niet eenvoudig door andere onderdelen van het programma worden gedeeld.

Eind vraag – betekenis in de c++

Toepasbaarheid

Compositie kan het beste gebruikt worden als wat je probeert te implementeren kan vergeleken worden met een boom structuur.

Het Composite-patroon biedt twee basistypen elementen die een gemeenschappelijke interface delen: eenvoudige bladeren (leaves) en complexe containers. Een container kan bestaan uit zowel bladeren als andere containers. Hierdoor kun je een geneste, recursieve objectstructuur opbouwen die lijkt op een boom.

Het implementeren van compositie

Compositie is een van de eenvoudigste relatietypen om te implementeren in C++. Het wordt meestal gemaakt als structs of klassen met gewone datamembers. Doordat deze datamembers rechtstreeks deel uitmaken van de struct/klasse, is hun levensduur gekoppeld aan die van de klasse-instantie zelf.

Composities die dynamisch geheugen moeten alloceren of vrijgeven, kunnen worden geïmplementeerd met pointer-datamembers. In dat geval moet de compositieklasse zelf verantwoordelijk zijn voor al het noodzakelijke geheugenbeheer (niet de gebruiker van de klasse).

In het algemeen geldt: als je een klasse kunt ontwerpen met compositie, doe dat dan ook. Klassen die met compositie zijn ontworpen, zijn eenvoudig, flexibel en robuust (in die zin dat ze netjes achter zichzelf opruimen).

Om als compositie te kwalificeren, moeten een object en een onderdeel de volgende relatie hebben:

· Het onderdeel (member) maakt deel uit van het object (de klasse)
· Het onderdeel (member) kan op een bepaald moment maar bij één object (klasse) horen
· Het bestaan van het onderdeel (member) wordt beheerd door het object (de klasse)
· Het onderdeel (member) weet niets van het bestaan van het object (de klasse)

(learncpp.com)

Bronnenlijst

- Alex. (2023, September 11). *23.2 — composition*. https://www.learncpp.com/cpp-tutorial/composition/
- GeeksforGeeks. (2026, September 16). *Composition in C++*. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/composition-in-c/
- Bhalla, N. (2023, 4 mei). What is Composition in C++? - Scaler Topics. *Scaler Topics*. https://www.scaler.com/topics/composition-in-cpp/ (geeft duidelijke voorbeelden over het onderwerp met best duidelijke uitleg)
- Kumar, A. K. (2024, 17 september). *Composite Pattern in C++*. modern.com. Geraadpleegd op 5 november 2026, van https://medium.com/@kamresh485/composite-pattern-in-c-d9b72997e5aa (staan duidelijk voorbeelden in voor html, best cool, ook geeft dit duidelijk voordelen en nadelen van compositie gebruiken en ook de nadelen van het zonder doen)
- Refactoring.Guru. (2026, 1 januari). *Composite in C++*. https://refactoring.guru/design-patterns/composite/cpp/example

Rolverdeling

Ryan (onderzoek en presenteren)
Liam (onderzoek en presentatie maken)
Myrthe (onderzoek en bronvermelding)
Darryl (powerpoint en presenteren)
Koen (schrijven)
Musawer (schrijven)
Rose (powerpoint, presenteren, GitHub-bewerken)